There's a similar check in `run_pending_task()` ❻, except this time you also need to check to see if there are any items on the local queue. If there are, you can take the front one and process it; notice that the local queue can be a plain `std::queue<>` ❶ because it's only ever accessed by the one thread. If there are no tasks on the local queue, you try the pool queue as before ❼.

This works fine for reducing contention, but when the distribution of work is uneven, it can easily result in one thread having a lot of work on its queue while the others have no work do to. For example, with the Quicksort example, only the topmost chunk would make it to the pool queue, because the remaining chunks would end up on the local queue of the worker thread that processed that one. This defeats the purpose of using a thread pool.

Thankfully, there is a solution to this: allow the threads to *steal* work from each other's queues if there's no work in their queue and no work in the global queue.

9.1.5. Work stealing

In order to allow a thread with no work to do to take work from another thread with a full queue, the queue must be accessible to the thread doing the stealing from `run_pending_tasks()`. This requires that each thread register its queue with the thread pool or be given one by the thread pool. Also, you must ensure that the data in the work queue is suitably synchronized and protected, so that your invariants are protected.

It's possible to write a lock-free queue that allows the owner thread to push and pop at one end while other threads can steal entries from the other, but the implementation of such a queue is beyond the scope of this book. In order to demonstrate the idea, we'll stick to using a mutex to protect the queue's data. We hope work stealing is a rare event, so there should be little contention on the mutex, and such a simple queue should therefore have minimal overhead. A simple lock-based implementation is shown here.

Listing 9.7. Lock-based queue for work stealing